

Zastosowane algorytmy i ich złożoność obliczeniowa

W ramach realizacji projektu dokonano analizy dostępnych metod rozwiązywania poszczególnych podproblemów oraz wyboru algorytmów najlepiej odpowiadających wymaganiom funkcjonalnym i wydajnościowym systemu. Przy doborze rozwiązań uwzględniono ich złożoność czasową oraz pamięciową, skuteczność działania oraz możliwość porównania różnych podejść do rozwiązania tego samego problemu obliczeniowego. Dodatkowo przeprowadzono analizę teoretycznej złożoności wybranych algorytmów, której wyniki przedstawiono na wykresach porównawczych.

Problem maksymalnego przepływu

W ramach implementacji algorytmu Min-Cost Max-Flow wykorzystano algorytmy **Forda–Fulkersona** oraz **Edmondsa–Karpa**, odpowiedzialne za wyznaczanie kolejnych ścieżek powiększających przepływ. Algorytmy te umożliwiają osiągnięcie maksymalnej wartości przepływu przy zachowaniu ograniczeń pojemnościowych sieci.

- **Ford–Fulkerson** – implementacja: **Filip Bąk**
- **Edmonds–Karp** – implementacja: **Kacper Dębski**

Analiza złożoności:

Algorytm	Złożoność czasowa	Złożoność pamięciowa
Ford–Fulkerson	$O(E \cdot F)$	$O(V + E)$
Edmonds–Karp	$O(V \cdot E^2)$	$O(V + E)$

gdzie:

- **V** oznacza liczbę wierzchołków,
- **E** oznacza liczbę krawędzi,
- **F** oznacza wartość maksymalnego przepływu.

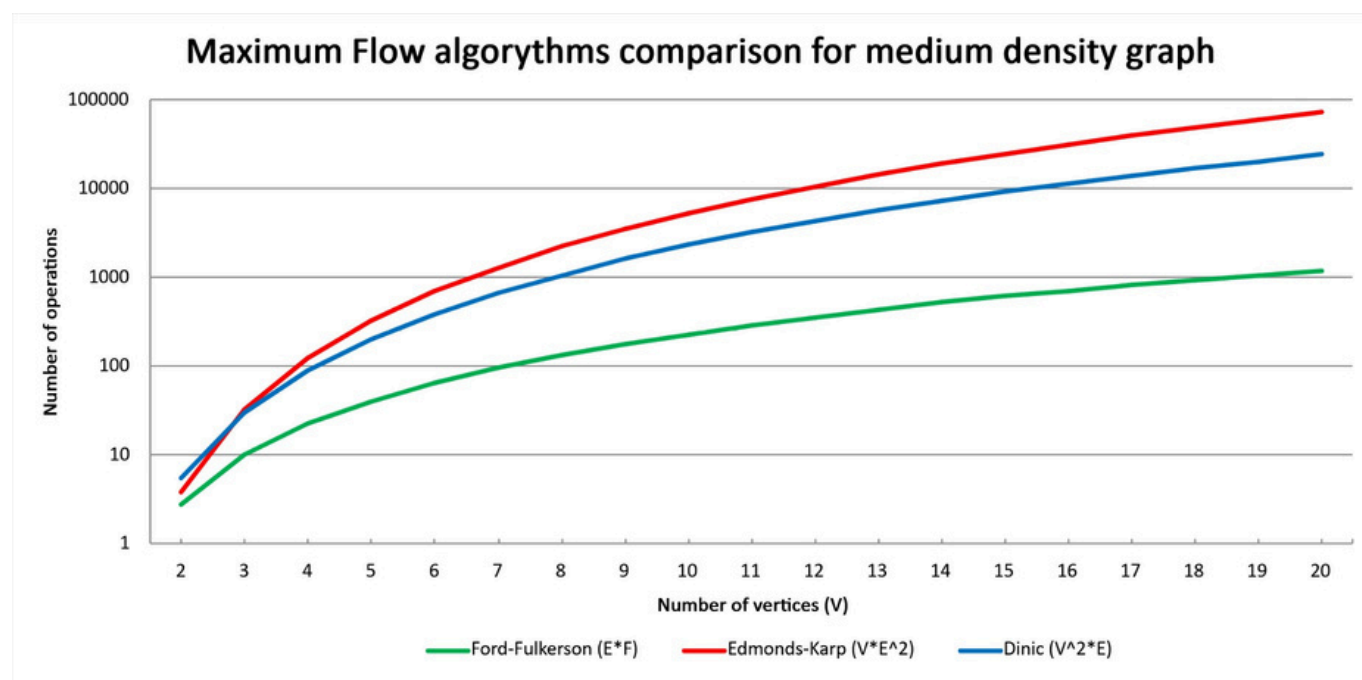
Jak przedstawiono na wykresie porównującym algorytmy maksymalnego przepływu, algorytm Forda–Fulkersona posiada złożoność zależną od wartości maksymalnego przepływu, natomiast Edmonds–Karp gwarantuje wielomianową złożoność czasową $O(V \cdot E^2)$. Uwzględnienie obu rozwiązań umożliwiło porównanie klasycznego podejścia augmentacyjnego z jego deterministyczną odmianą wykorzystującą przeszukiwanie BFS.

Uzasadnienie poprawności

Algorytmy Forda–Fulkersona oraz Edmondsa–Karpa stanowią podstawę procedury zwiększania przepływu w sieci. W każdej iteracji wyznaczana jest ścieżka powiększająca w sieci rezydualnej, a następnie przepływ jest zwiększany z zachowaniem ograniczeń pojemności oraz zasady zachowania przepływu w każdym wierzchołku. Proces ten trwa do momentu, gdy nie istnieje już żadna ścieżka powiększająca pomiędzy źródłem a ujściem. Z twierdzenia Forda–Fulkersona wynika, że w takim przypadku otrzymany przepływ jest maksymalny.

W konsekwencji algorytmy Forda–Fulkersona oraz Edmondsa–Karpa poprawnie wyznaczają maksymalny przepływ w sieci. Otrzymany przepływ stanowi następnie podstawę działania algorytmu Min-Cost Max-Flow odpowiedzialnego za minimalizację całkowitego kosztu transportu. Dzięki wykorzystaniu najkrótszych ścieżek

kosztowych podczas kolejnych augmentacji możliwe jest jednoczesne osiągnięcie maksymalnej wartości przepływu oraz minimalizacja całkowitego kosztu transportu. W rezultacie otrzymane rozwiązanie spełnia zarówno ograniczenia przepustowości sieci, jak i wymagania optymalizacji kosztowej.



Problem minimalnego kosztu

W celu rozwiązania problemu minimalizacji kosztu transportu zastosowano algorytmy **Bellmana–Forda** oraz **d’Esopo–Pape**, wykorzystywane do wyznaczania najkrótszych ścieżek kosztowych w algorytmie *Min-Cost Max-Flow*.

- **Bellman–Ford** – implementacja: **Samuel Kępski**
- **d’Esopo–Pape** – implementacja: **Filip Bąk**

Analiza złożoności:

Algorytm	Złożoność czasowa	Złożoność pamięciowa
Bellman–Ford	$O(V \cdot E)$	$O(V)$
d’Esopo–Pape	$O(V \cdot E)$ (pesymistycznie)	$O(V + E)$

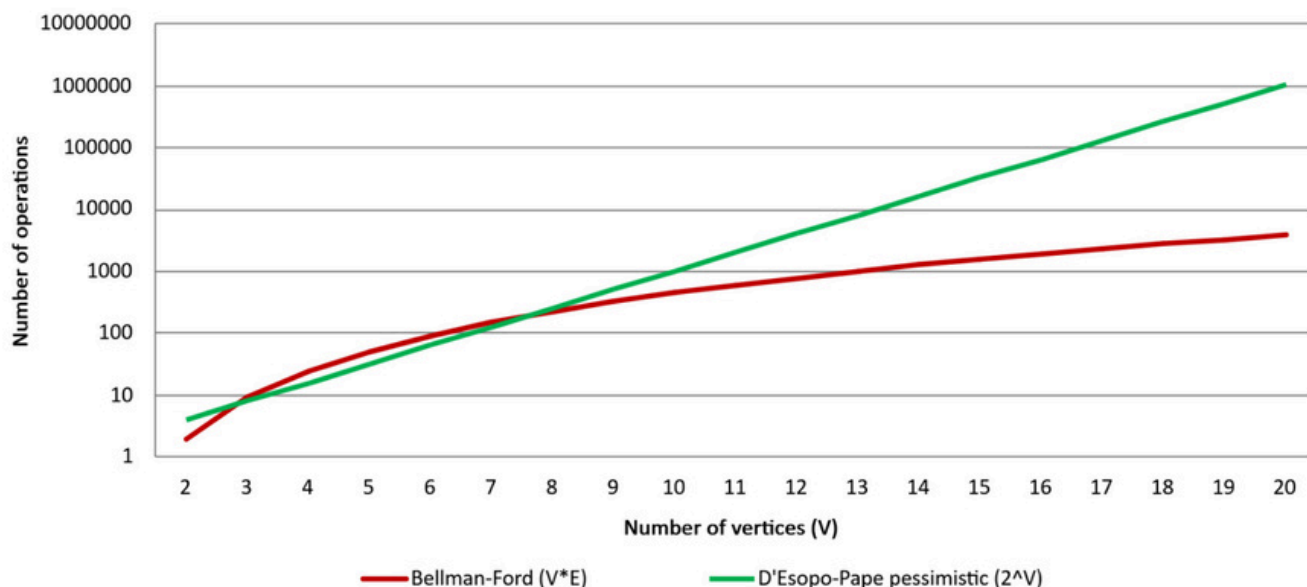
Zarówno algorytm Bellmana–Forda, jak i d’Esopo–Pape posiadają pesymistyczną złożoność czasową $O(V \cdot E)$. Należy jednak podkreślić, że algorytm d’Esopo–Pape w praktyce często osiąga znacznie lepsze czasy wykonania dla wielu typowych klas grafów dzięki efektywniejszemu przetwarzaniu wierzchołków. Z tego względu oba rozwiązania zostały uwzględnione w celu porównania ich właściwości teoretycznych oraz praktycznej wydajności.

Uzasadnienie poprawności

Algorytm Bellmana–Forda wyznacza najkrótsze ścieżki poprzez wielokrotną relaksację wszystkich krawędzi grafu. Po wykonaniu $|V|-1$ iteracji każda najkrótsza ścieżka została uwzględniona, co gwarantuje poprawność wyniku. Algorytm d’Esopo–Pape również opiera się na relaksacji krawędzi i zachowuje niezmiennik

poprawnych odległości od źródła. W rezultacie oba algorytmy wyznaczają poprawne najkrótsze ścieżki wykorzystywane w procedurze Min-Cost Max-Flow.

Min-Cost Flow algorithms comparison for medium density graph



Problem otoczki wypukłej

Dla problemu wyznaczania otoczki wypukłej zbioru kopali zaimplementowano trzy klasyczne algorytmy geometrii obliczeniowej:

- **Jarvis March** – implementacja: **Kacper Dębski**
- **Andrew's Monotone Chain** – implementacja: **Samuel Kępski**
- **Graham Scan** – implementacja: **Filip Bąk**

Analiza złożoności:

Algorytm	Złożoność czasowa	Złożoność pamięciowa
Jarvis March	$O(n \cdot h)$	$O(h)$
Graham Scan	$O(n \log n)$	$O(n)$
Andrew's Monotone Chain	$O(n \log n)$	$O(n)$

gdzie:

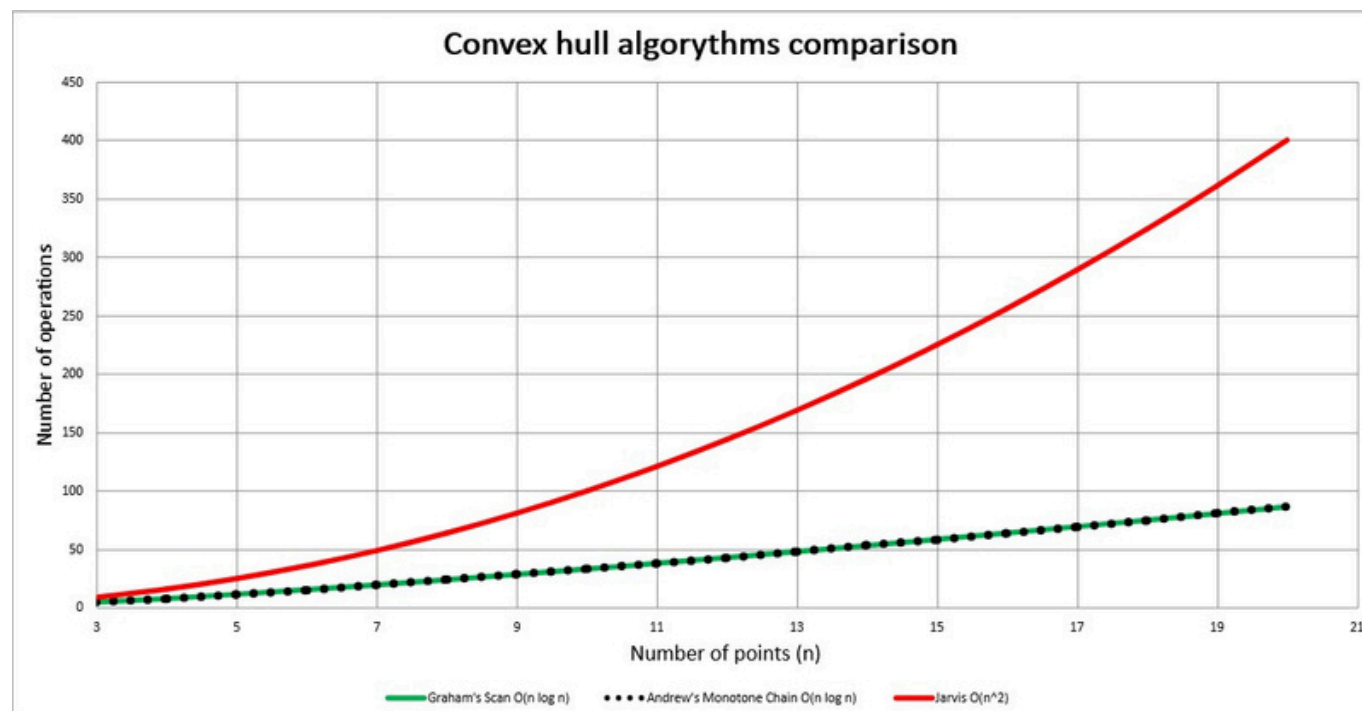
- n oznacza liczbę punktów,
- h oznacza liczbę punktów należących do otoczki.

Analiza złożoności przedstawiona na wykresie pokazuje, że algorytmy Graham Scan oraz Andrew's Monotone Chain posiadają złożoność rzędu **$O(n \log n)$** , dzięki czemu skalują się znacznie lepiej wraz ze wzrostem liczby punktów niż algorytm Jarvis March. Złożoność algorytmu Jarvis March wynosi **$O(n \cdot h)$** , gdzie h oznacza liczbę punktów należących do otoczki wypukłej. W najgorszym przypadku, gdy wszystkie analizowane punkty należą do otoczki, złożoność ta upraszcza się do **$O(n^2)$** . Zastosowanie trzech różnych metod umożliwiło porównanie

ich zachowania dla tego samego problemu geometrycznego oraz ocenę wpływu złożoności obliczeniowej na wydajność rozwiązania.

Uzasadnienie poprawności

Wszystkie zaimplementowane algorytmy konstruuja wielokąt zawierający wszystkie analizowane punkty oraz odrzucają punkty znajdujące się wewnątrz otoczki. Jarvis March kolejno wybiera skrajne punkty otoczki, natomiast Graham Scan i Andrew's Monotone Chain wykorzystują orientację trójek punktów do eliminacji punktów niespełniających warunku wypukłości. Ostatecznie zwracany zbiór punktów tworzy poprawną otoczkę wypukłą analizowanego zbioru.



Problem najgłośniejszego krasnoludka

W ramach problemu wyszukiwania najgłośniejszego krasnoludka na zadanym odcinku granicy zaimplementowano dwa rozwiązania:

- **Brute Force** – implementacja: **Filip Bąk**
- **Segment Tree** – implementacja: **Kacper Dębski**

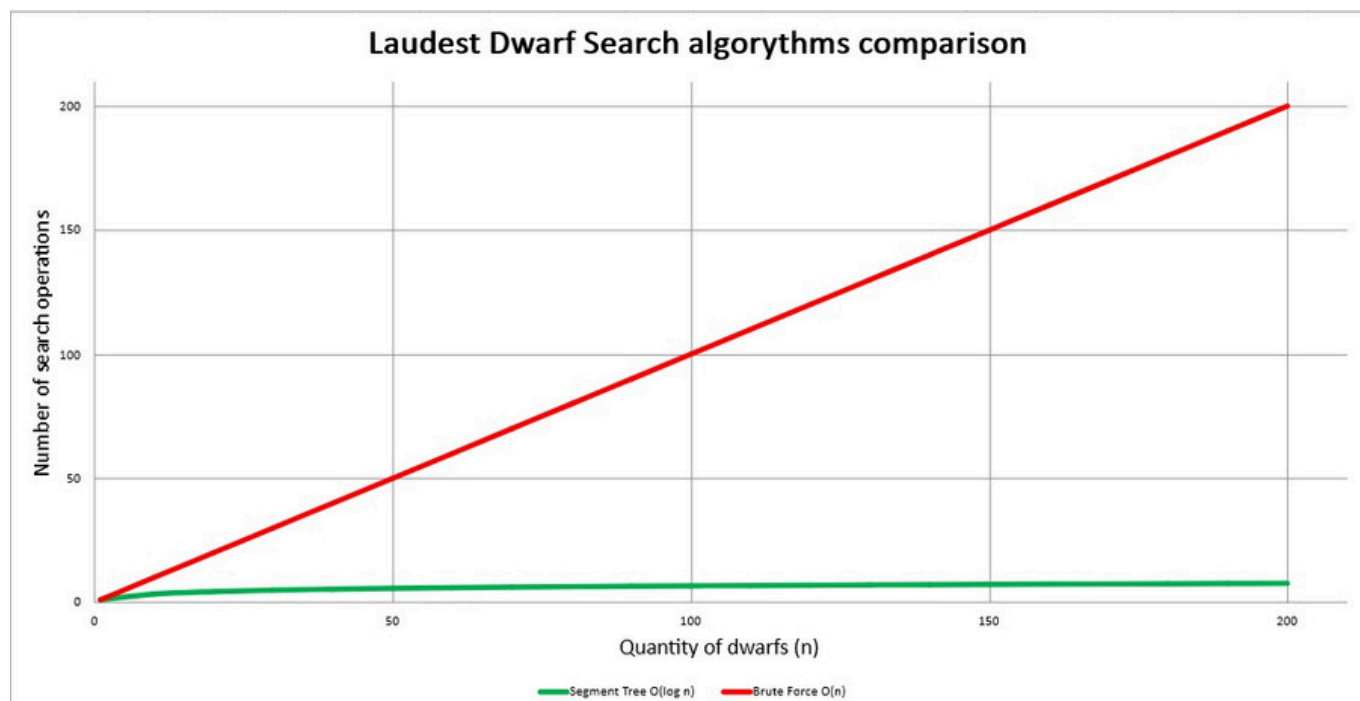
Analiza złożoności

Algorytm	Budowa	Zapytanie	Pamięć
Brute Force	$O(1)$	$O(n)$	$O(1)$
Segment Tree	$O(n)$	$O(\log n)$	$O(n)$

Jak pokazuje wykres porównawczy, metoda Brute Force wymaga liniowej liczby operacji względem rozmiaru danych ($O(n)$), natomiast drzewo przedziałowe Segment Tree pozwala wykonywać zapytania w czasie logarytmicznym ($O(\log n)$). Różnica wydajności staje się coraz bardziej widoczna wraz ze wzrostem liczby krasnoludków i liczby wykonywanych zapytań.

Uzasadnienie poprawności

Drzewo przedziałowe przechowuje w każdym węźle maksimum odpowiadające mu przedziału. Ponieważ wartość każdego węzła jest wyznaczana jako maksimum jego dzieci, maksimum dla dowolnego zakresu można wyznaczyć przez połączenie odpowiednich fragmentów drzewa. Z własności maksimum wynika, że zwrócona wartość jest największym elementem analizowanego przedziału.



Optymalizacja i przechowywanie danych

Za realizację modułu optymalizacji i przechowywania danych odpowiadał **Samuel Kępski**. W jego zakresie znalazła się implementacja:

- interfejsu graficznego użytkownika (GUI),
- obsługi przesyłania plików,
- struktur JSON oraz integracji wszystkich modułów systemu,
- algorytmu kodowania Huffmana.

Analiza złożoności algorytmu Huffmana

Operacja	Złożoność czasowa	Złożoność pamięciowa
Budowa drzewa Huffmana	$O(n \log n)$	$O(n)$
Kodowanie danych	$O(m)$	$O(n)$
Dekodowanie danych	$O(m)$	$O(n)$

gdzie:

- n oznacza liczbę różnych symboli,
- m oznacza długość kodowanego tekstu.

Algorytm Huffmana został wybrany ze względu na możliwość bezstratnej kompresji danych przy zachowaniu stosunkowo niewielkich kosztów obliczeniowych. Rozwiązanie to pozwala ograniczyć ilość pamięci potrzebnej do przechowywania wyników analiz oraz konfiguracji systemu.

Uzasadnienie poprawności

Algorytm Huffmana buduje prefiksowy kod binarny na podstawie częstości występowania symboli. Każdy symbol odpowiada dokładnie jednemu liściowi drzewa kodowego, a żaden kod nie jest prefiksem innego kodu. Dzięki temu zakodowane dane mogą zostać jednoznacznie odtworzone bez utraty informacji, co gwarantuje bezstratność kompresji.